

G08 · Sleep

G08 · Sleep — 자고 일어나 정리하기 (핵심)

이 권은 이 계열의 마지막 아이디어, **Sleep** 을 다룬다 (Language Models Need Sleep, arXiv:2606.03979, Behrouz et al.). 지금까지의 권들은 “메모리를 어떻게 읽고 쓰고 갱신 하나” 를 바꿨다. 이 권은 한 걸음 물러나 **모델의 하루 일과 자체** 를 바꾼다 — 깨어서 배우고 (wake), 자면서 정리한다 (sleep). 왜 필요한지, 자는 동안 정확히 무슨일이 일어나는지, 그리고 그것이 서빙 시스템에 무엇을 요구하는지를 수식없이 그림과 비유로 먼저 이해하는 것이 목표다.

1. 문제: 계속 배우면 예전 걸까먹는다

앞 권들의 메모리는 “읽으면서 계속 자기를 학습시키는 작은 모델” 이었다. 새토큰이 오면 그 토큰에 맞춰 메모리 파라미터를 조금 고쳐 쓴다. 여기에 근본적인 함정이 하나 있다.

메모리의 크기 (파라미터 수) 는 **고정** 이다. 그런데 새 지식을 계속 밀어 넣는다. 그릇은 그 대로인데 물을 계속 붓는 셈이다. 결국 새 내용이 옛내용을 **덮어쓴다**. 이것이 **catastrophic forgetting** (치명적 망각, 줄여서 CF) 이다 — 방금 배운 걸 넣느라 예전에 잘 알던 걸 잃는 현상.

직관 CF 는 “기억력이 나빠서” 가 아니라 **자리 (capacity) 가 부족해서** 생긴다. 유한한 파라미터에 무한히 배우면, 새것을 위해 옛것을 지울 수밖에 없다. 그래서 해법도 “더 조심히 덮어쓰자” 가 아니라 “덮어쓰기 전에 옮겨놓고, 자리를 늘리자” 가 된다.

앞 권 (G07 Nested Learning) 은 이 문제를 **미루는** 데까지 갔다. 빠르게 갱신되는 메모리와 느리게 갱신되는 메모리를 여러 층으로 쌓으면 (CMS, Continuum Memory System), 덮어쓰기가 층마다 다른 속도로 일어나 CF 가 늦게 온다. 하지만 늦출 뿐, 없애지는 못한다. 모든 층의 갱신주기가 겹치는 순간 결국 덮어쓴다.

또 하나의 이상한 점: 지금까지 우리는 학습을 “train time 에 하고 test time 에 멈춘다” 고 나눠 왔다. 그런데 배포된 뒤에도 계속 배우는 모델 (continual learner) 에게 이 구분이 맞나? 계속 배운다면 “훈련이 끝나는 시점” 이 없다. Sleep 논문의 첫 주장이 바로 이것이다: **계속 배우는 모델에게는 train time 도 test time 도 없다.**

2. 해법의 씨앗: 뇌는 잔다

사람은 낮에 공부한 걸 밤에 자면서 장기 기억으로 정리한다. 뇌과학에서 이것 **memory consolidation** (기억 공고화) 이라 부른다. 낮 동안 해마 (hippocampus) 에 임시로 담아 둔 것을, 자는 동안 대뇌피질 (neocortex) 의 안정된 장기 저장소로 옮긴다.

Sleep 논문은 이 그림을 그대로 LLM 에 이식한다. 모델의 삶을 두 phase 의 반복으로 재정의한다.

- **Wake (active) phase:** 새 입력을 받아 처리하고, 빠른 메모리에 계속 쓴다. 우리가 아는 inference + 앞 권들의 test-time 학습.

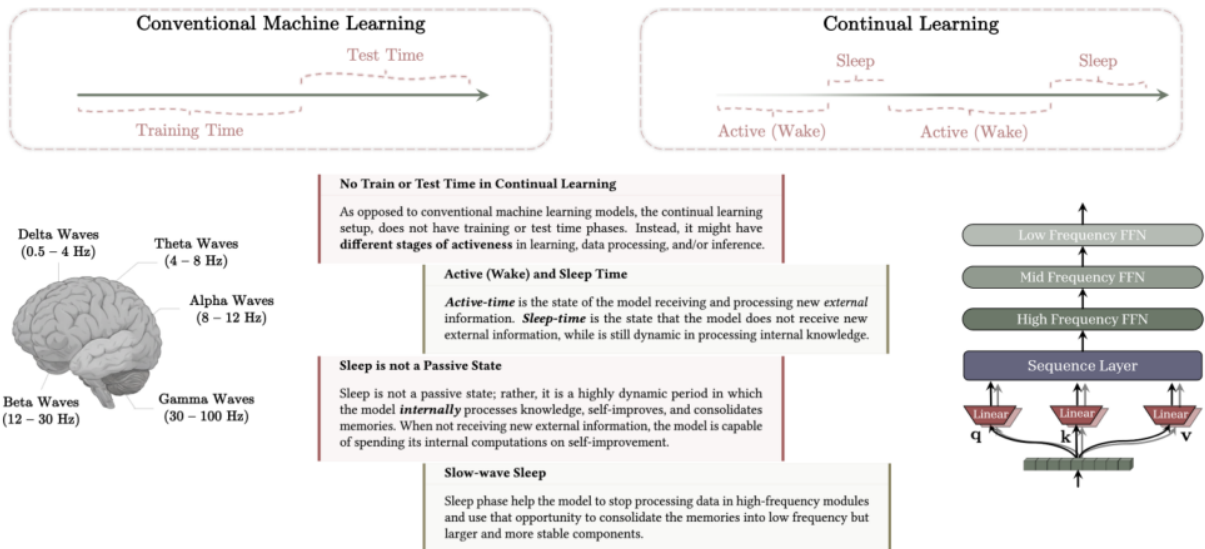


그림 G08-1. Sleep 이 다시 그린 모델의 lifecycle. 왼쪽 (Conventional Machine Learning) 은 모델의 수명을 training time 과 test time 으로 나눈다. 오른쪽 (Continual Learning) 에는 그 구분이 없고, 입력을받는 Active(Wake) 구간과 입력을 끊고 내부를 정리하는 Sleep 구간이주기적으로 교대한다. 오른쪽 아래는 그 정리를 수행하는 Low/Mid/High Frequency FFN 사슬 (뒤에서 다룰 CMS 백본) 이다. 출처: Behrouz et al., Sleep(arXiv:2606.03979) Fig.1 — 원저자 그림.

- **Sleep phase:** 새 입력을 끊고, 안으로 조용히 계산해 기억을정리하고 자기를 개선한다.

비유 Wake 는 낮의 공부, Sleep 은 밤의 복습 · 정리다. 낮에는 노트 (빠른 메모리) 에마구 적고, 밤에는 그 노트를 다시 훑어 중요한 것만 교과서 (느린 메모리) 에 옮겨 적고 노트는 비운다. 다음 날 노트는 다시 깨끗한 채로 시작한다.

핵심 Sleep 은 앞 권들의 갱신 규칙 (어떻게 쓰나) 을 바꾸지 않는다. 바꾸는 것은 그 갱신들이 살아가는 **시간 구조 (lifecycle)** 다. G07 이 “여러 속도의메모리를 공간축에 쌓았다” 면, Sleep 은 거기에 **시간축**을 하나 더놓아 “자는 시간” 을 만든 것이다.

sleep 이 하는 일은 두 가지다. 이 권의 나머지는 이 둘을 하나씩 푼다.

1. **Consolidation** — 빠른 메모리가 덮어써지기 **직전에**, 그지식을 느린 메모리로 “승격 저장” 하고 빠른 메모리를 비운다.
2. **Dreaming** — 스스로 연습문제 (합성 데이터) 를 만들어, 그중 쓸모있는 것만 골라 다시 학습한다.

3. sleep 은 언제오는가

먼저 “언제 자나” 부터. 이건 학습되지 않는다. 그냥 **스케줄에 고정**되어 있다.

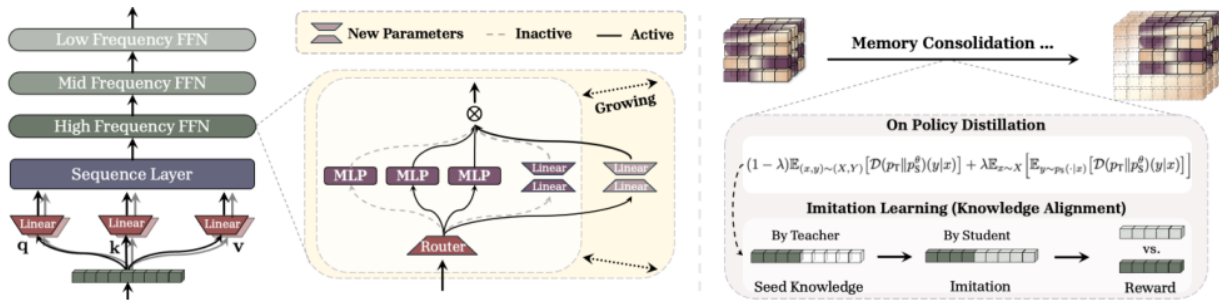


그림 G08-2. Memory Consolidation 개관. 모델은 먼저 새 low-rank 파라미터 (새 expert 슬롯)를 켜서 자리를 늘린 뒤 (왼쪽·가운데), Knowledge Seeding으로 고주파 메모리의 지식을 저주파 메모리로 옮긴다. 오른쪽은 그이전을 실제로 수행하는 두 항 — teacher가 만든 데이터와 student 스스로의 rollout을 섞는 distillation, 그리고 teacher를 흉내 내도록 보상을 주는 Imitation Learning. 출처: Behrouz et al., Sleep(arXiv:2606.03979) Fig.2 — 원저자 그림.

앞 권에서 봤듯 CMS의 각 메모리 블록은 자기만의 갱신 주기를 가진다. 실험기준 사다리는 **1k → 5k → 10k 토큰**이다. 즉 High-frequency 블록은 1,000 토큰마다, Mid는 5,000 토큰마다, Low는 10,000 토큰마다 한 번 갱신된다.

sleep(정리 작업)은 바로 이 **갱신 경계에서** 일어난다. 어떤 블록이자기 갱신 시점에 도달하면, 그 블록을 덮어쓰기 **직전에** 내용을다음 느린 블록으로 먼저 옮긴다.

직관 “곧 덮어쓸 노트가 있다 → 덮기 전에 교과서로 옮겨라.” sleep은 알람이아니라 **마감 직전 백업**이다. 빠른 블록은 자주 마감(1k)이 오고느린 블록은 드물게(10k) 온다. 그래서 빠른→느린 이전은 여러 번, 느린블록이 받는 건 가끔이다.

주기가 겹쳐 있으므로 이전은 **다대일**이다. 느린 블록이 한 번갱신되는 동안 빠른 블록은 약 10 번 갱신되고, 따라서 빠른→느린 이전이그동안 여러 번 일어난다. 같은 고정 크기의 느린 메모리에 여러 번 반복해서써 넣는 것 — 여기가 바로 CF가 터질 자리다. 그래서 다음 절의 “자리늘리기”가 필요해진다.

4. 일 (1): Consolidation — 덮어쓰지 말고, 위로 승격저장

Consolidation 한 사이클의 큰 그림이 아래 그림이다. 순서는 이렇다: (a) 자리를 늘리고 → (b) 지식을 위로 옮기고 → (c) 쓰는 법을가르치고 → (d) 빠른 메모리를 비운다.

4a. 자리 늘리기 (parameter expansion): 덮어쓰지 말고키워라

핵심 발상: 옛 지식 **위에** 새 지식을 쓰지 말고, 옆에 **새슬롯을 열어** 거기에만 쓴다. 그러면 옛 지식은 물리적으로 건드려지지않는다.

받는 쪽 느린 블록은 sparse **MoE**(Mixture-of-Experts)로 되어 있다— 여러 개의 작은 전문가 (expert) 중 필요한 것만 골라 쓰는 구조. consolidation을 받을 때마다 **새 expert 하나**를 추가한다. 이 새 expert는 작게 만든다: **low-rank MLP**다.

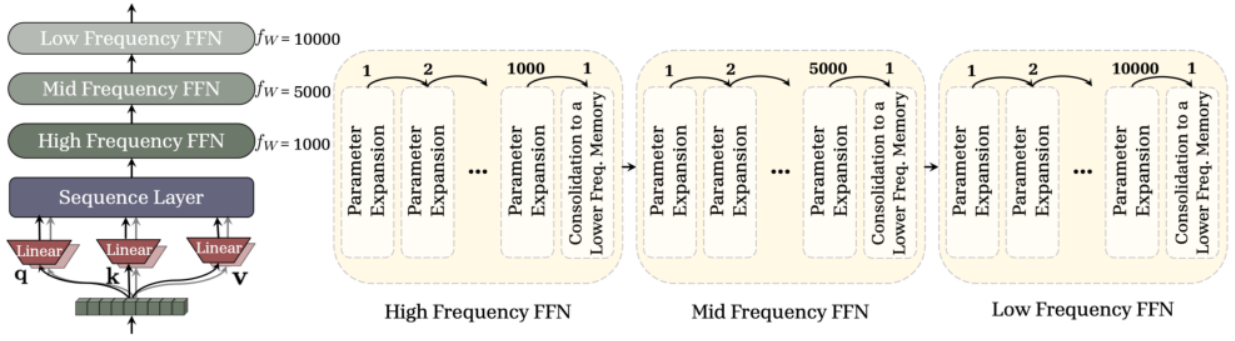


Figure 7: Multi-frequency memory hierarchy. Updates enter the High-Frequency FFN via repeated Parameter Expansion; when the window f_W expires, knowledge is Consolidated to the Mid- and then Low-Frequency FFNs (1k→5k→10k).

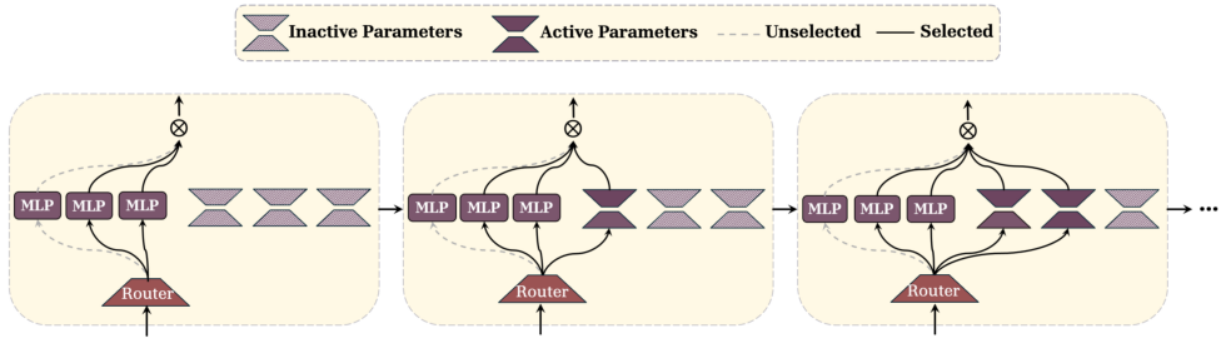


그림 G08-3. Routed expert 로 하는 consolidation. Sleep 사이클을 거치며 (왼→오) router 가 소수의 expert(칠해진 것) 만 골라 갱신한다. 새 지식은 새로 켜 expert 에만 들어가고 나머지는 건드리지 않으므로, 옛 지식이 덮여지지 않는다. 출처: Behrouz et al., Sleep(arXiv:2606.03979) Fig.8 —원저자 그림.

직관 교과서를 통째로 새로 쓰는 게 아니라, 뒤에 **새 페이지 한 장을 끼워** 거기에만 오늘 배운 걸 적는다. 옛 페이지는 그대로 두니 안 지워진다. sleep 이 지날 때마다 느린 메모리는 페이지가 조금씩 늘어난다.

새 expert 는 두 개의 작은 행렬로 만든다. 이 부분은 크기 계산이 중요하니 기호를 정확히 풀어 둔다.

$$A \in \mathbb{R}^{d \times d_{\text{low}}}, \quad B \in \mathbb{R}^{d_{\text{low}} \times d}, \quad d_{\text{low}} \ll d$$

기호 풀이 d = 모델의 hidden dimension(예: 수천). d_{low} = 새 expert 의 작은 내부 차원 (실험에선 64). A = 입력을 작은 차원으로 눌러담는 행렬. B = 작은 차원을 다시 원래 크기로 퍼는 행렬. $\ll$ = “훨씬 작다”.

이 식은 ~라는 뜻. 새로 옮겨 오는 지식은 오직 이 두 작은 행렬 안에만 저장된다. 저장에 드는 값의 개수는 총 $2 d d_{\text{low}}$ 개뿐 — d^2 이 아니라 그것의 아주 작은 일부다.

구현상 한 가지 실전 요령이 있다. 실행 중에 tensor 크기를 실제로 늘리면 컴파일된 그래프 · kernel 이 다 깨진다. 그래서 **미래에 열릴 expert 들을 처음부터 다 만들어 놓고 mask**

로 꺼 둔다. “expert 추가” 는 사실 mask 를 하나 여는 것이다. shape 는 항상 고정이라 시스템이 안정적이다.

시스템 모델링 관점 consolidation 1 회가 만드는 새 state 는 **low-rank expert 하나** = $2 d d_{low}$ 개의 값이다 ($d_{low}=64$, 블록 5 개규모의 실험). 이것은 KV cache 와 성질이 다르다. KV cache 는 토큰마다 append 되어 $O(L)$ 로 자라지만 (L =시퀀스 길이), expert delta 는 **sleep** 마다 하나씩, 고정 크기로 자란다. shape 는 mask 덕에 정적이므로 재컴파일이 없다. 서버 관점에서 이걸 KV 문제가 아니라 **per-user weight-delta(LoRA adapter)** 서버 문제에 가깝다 — adapter 의 버전 관리 · routing · eviction 이 세션 관리의 어휘가 된다.

4b. 지식 옮기기 (Knowledge Seeding): 위로 가는 distillation

새 슬롯을 열었으니, 이제 빠른 블록의 지식을 그 슬롯으로 옮겨야 한다. 방법은 distillation(증류) 인데, 방향이 거꾸로다.

보통 distillation 은 큰 teacher → 작은 student 로 지식을 압축한다. 여기서 **작은 teacher → 큰 student** 로 간다. 그래서 이름이 **Knowledge Seeding(KS)**, 별칭 upward distillation 이다. teacher 와 student 가 같은 모델의 두 시점일 때는 **Self-Knowledge Seeding** 이라 부른다.

기호 풀이 teacher = 자리를 늘리기 전의 모델 — 빠른 블록이 아직 덮어쓰이지기 전, 옮길 지식을 온전히 들고 있는 상태. student = 자리를 늘리고 빠른 블록을 이미 새 chunk 로 갱신한 후의 모델 — 대신느린 블록에 빈 새 expert 를 하나 받은 상태.

옮기는 방식: student 의 출력이 teacher 의 출력과 같아지도록 당긴다. 단, student 에서 움직일 수 있는 **파라미터는 새 expert 하나뿐** (나머지는 전부 동결). 그래서 최적화가 성공하면, 빠른 블록에서 곧 사라질 지식인 느린 블록의 새 expert 안에 다시 조립된다. 이것이 “덮어쓰기 직전에 consolidate 한다” 의 정확한 실체다.

여기엔 잔재주가 하나 있다. teacher 가 미리 만든 고정 데이터로만 student 를 가르치면, student 가 자기 능력을 다 못 쓴다 (student 가 더 크니까). 그래서 **student 가 스스로 만든 문장(on-policy rollout)** 위에서도 교정을 받게 쉰다. 이 쉰 방식이 GKD(Generalized Knowledge Distillation) 다.

$$\mathcal{L}_{GKD} = (1 - \lambda_{on}) \cdot (\text{teacher 데이터 위 오차}) + \lambda_{on} \cdot (\text{student 자기 문장 위 오차})$$

기호 풀이 $\lambda_{on} \in [0, 1]$ = on-policy 비중. 0 이면 teacher 가 준 문장으로만 배우고, 1 이면 student 가 만든 문장으로만 배운다. “오차” = teacher 와 student 의 다음-토큰 확률분포가 얼마나 다른가 (divergence).

이 식은 ~라는 뜻. teacher 의 모범답안과 student 의 실전 답안을 λ_{on} 비율로 섞어 가르친다. 그래야 student 가 teacher 흉내에 갇히지 않고 자기 크기를 살린다.

한계 이 옮기기는 **lossy**(손실 있음) 다. 지식을 작은 low-rank 슬롯에 압축해 담으므로 원본 그대로가 아니라 “요약된 추상”이 저장된다. 무엇이 보존되고 무엇이 버려지는지에 대한 이론적 보장은 논문에 없다.

4c. 쓰는 법 가르치기 (Learning to Imitate)

옮겨 담기만으로는 부족했다는 게 논문의 관찰이다. student가 지식에 접근은 하는데 쓸 줄을 몰라, teacher의 실력을 약하게만 흉내 냈다. 아는 것과 쓰는 것은 다르다.

그래서 짧은 RL 단계를 붙인다: **Learning to Imitate**(LTI). teacher가 만든 글의 앞부분 (prefix)을 student에게 주고 뒤를 이어 쓰게 한뒤, 잘 이었으면 보상을 준다.

$$r = \rho \cdot r_{\text{sem}} + (1 - \rho) \cdot r_{\text{abs}}$$

기호 풀이 r = student가 받는 총 보상. r_{sem} = 의미가 teacher 원본과 같으면 1, 다르면 0(고정된 외부 reward model이 판정). r_{abs} = 글자 단위로 얼마나 비슷한가 (Levenshtein 편집거리기반, 0~1). $\rho \in [0, 1]$ = 둘을 섞는 비율.

이 식은 ~라는 뜻. “의미도 맞고 표현도 비슷하게” teacher를 따라 하면 높은 점수. 의미 점수와 글자 점수를 ρ 로 저울질한다.

4d. 다 옮겼으면 비운다 (synaptic-pruning reset)

옮기기가 끝나면 마지막 동작: 빠른 블록에 그동안 쌓였던 임시 expert들을 전부 **reset**(초기화) 한다. 자리를 되돌려 다음 wake를 위해 비운다.

비유 밤에 노트 내용을 교과서로 다 옮겼으니, 노트를 지우개로 지워 내일 아침 깨끗하게 시작한다. 뇌가 불필요한 연결을 처내는 synaptic pruning에 대응한다.

핵심 순 효과는 하나의 규칙으로 요약된다 — 빠른 메모리는 작고 잘 지워지게 (plastic) 유지되고, 느린 메모리는 정해진 한도 안에서 단조성장하며, 오직 새 expert를 통해서만 늘어난다. inference 어휘로는 “eviction 전에 반드시 상위 계층으로 write-back 하는 cache 정책”이다.

5. 여러 속도의 메모리 계층 (multi-frequency)

지금까지의 이야기를 한 장으로 모으면 아래 그림이다. 갱신 속도가 다른 FFN(메모리 블록)들이 사슬로 쌓여 있고, 빠른 블록은 자주 팽창·정리되며, 자기 window가 만료될 때 한 단계 느린 블록으로 consolidation을 흘려보낸다.

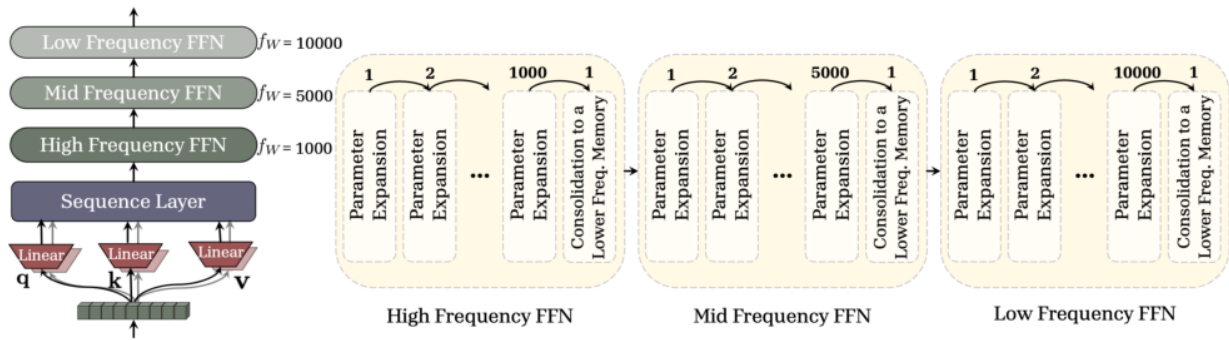


그림 G08-4. Multi-frequency memory hierarchy. 왼쪽은 Sequence Layer 위에 갱신 주기가 다른 FFN 사슬 (High/Mid/Low, 주기 1k/5k/10k 토큰) 이 쌓인 CMS 백본. 오른쪽은 각 FFN 이 Parameter Expansion 을 반복하다 window 가 만료되면 다음 저주파 FFN 으로 Consolidation 을 넘기는 스케줄 (1k → 5k → 10k). 빠른 블록이 여러 번 팽창 · 정리되는 동안 느린 블록은 한번만 받는 주기 중첩이 눈에 보인다. 출처: Behrouz et al., Sleep(arXiv:2606.03979) Fig.7 — 원저자 그림.

시스템 모델링 관점 이 그림은 그대로 **storage-tier 설계 힌트**로 읽힌다. 고주파 (1k) 블록은 accumulator 를 자주 건드리므로 연산에 가까운 빠른 메모리에 상주해야 한다. 저주파 (10k) 블록은 수천 토큰에 한 번만 write 하므로 더 느린 계층에 두어도 write 비용이 상각된다. **update frequency 사다리 = 메모리 계층 사다리**. 각 블록의 state 크기는 “블록 파라미터 + 같은 크기의 accumulator” 한 쌍으로 잡으면 된다.

6. 일 (2): Dreaming — 스스로 연습문제를 만든다

sleep 의 두 번째 일. REM 수면에 대응하는 **Dreaming** 은, 모델이 자기실력을 올려 줄 **합성 데이터 (dream)** 를 스스로 만드는 법을 배우는 과정이다.

비유 시험을 잘 보려고 스스로 **예상 문제를 만들어** 풀어 보는 것. 그중 실제로 도움 된 문제 유형은 더 자주 만들고, 쓸모없던 건 버린다. 시간이 지나면 “잘 듣는 연습문제” 를 만드는 감각이 는다.

절차는 다섯 단계다.

1. **생성**. 주어진 task 맥락에서 dream(합성 텍스트) 여러 개를 뽑는다. 이때 MoE router 가 정상 선택에 더해 **무작위 expert 하나를 일부러 더 켜다** — 무관한 지식을 살짝 섞어, 평소 안 보이던 기억사이 연결을 표면화하는 novelty 주입이다.
2. **선별**. 각 dream 이 학습에 얼마나 도움 될지를 gradient 로 점수매겨, 대부분을 버리고 상위 몇 개 + 다양성용 무작위 몇 개만 남긴다.
3. **적용**. 남은 dream 마다 모델의 **독립 사본**을 하나 만들어 그 dream 으로 살짝 fine-tune(LoRA) 한다.
4. **보상**. fine-tune 된 사본이 실제 task 에서 원래보다 좋아졌으면 그 dream 생성에 1 점, 아니면 0 점.
5. **강화**. 이 점수로 “좋은 dream 을 만드는 습관” 을 강화한다 (ReST[^]EM 방식).

주의 Consolidation 과 Dreaming 의 순서가 설계 포인트다. 먼저 consolidation 으로 갓 배운 취약한 지식을 새 expert 에 격리해 둔 다음에 dreaming 을 돌린다. 그래야 dreaming 의 반복 학습이 그지식을 다시 지울 위험이 줄어든다. 논문은 이 순서가 더 안전하다는 **가설과 실험**을 제시할 뿐, 침식이 불가능하다고 보장하지는 않는다.

한계 Dreaming 은 **task-aware** 다. 즉 “어떤 맥락인지 (x_{ctx})” 와 “성능을 어떻게 썰지 (τ)” 가 sleep 중에 주어져야 한다. 평가 척도 τ 가 없는 완전 비지도배포에서는 4 단계의 “좋아졌나?” 보상을 무엇으로 대체할지가 공백이다.

7. 한 번의 sleep, 전체 흐름

앞의 조각들을 실행 순서로 이으면 sleep 한 사이클은 이렇다. 어떤 빠른블록이 갱신 경계에 닿았을 때:

1. **teacher 저장** — 현재 상태를 teacher 로 스냅샷.
2. **자리 늘리기** — 다음 느린 블록에 새 low-rank expert 하나를 켜.
3. **빠른 블록 갱신** — 빠른 블록을 새 chunk 로 갱신한다 (이 시점 모델이 student).
4. **연습 데이터 생성** — teacher 에서 문장을 뽑아 distillation 용 · LTI 용 데이터를 만든다.
5. **Knowledge Seeding** — 새 expert 만 학습 대상으로, 지식을 위로 옮긴다 (GKD + LTI).
6. **reset** — 빠른 블록의 옛 임시 expert 들을 비운다.
7. **Dreaming** — task 가 주어져 있으면 6 절의 5 단계 loop 를 돈다.
8. **wake 재개** — 갱신된 모델이 다음 입력을 받는다.

8. 시스템 모델링 관점: sleep 은 서빙에 붙는 background job

이 논문의 시스템 함의는 이 계열의 어느 논문보다 크다. 앞 권들은 layer 를 바꿨지만, Sleep 은 배포 형태를 바꾼다.

시스템 모델링 관점 (핵심 정리). - **inference** 가 더 이상 **forward-only** 가 아니다. wake 중 각 메모리 블록은 토큰마다 optimizer 오차를 계산하고 (backward 류 연산 필요), 파라미터 크기의 accumulator 를 유지하며, 주기마다 weights 에 write 한다. KV cache 의 append-only 트래픽과 달리 이건 **read-modify-write(RMW)** 트래픽이고, 이 대역폭이 cost model 의 새 항이 된다. - **state** 의 종류가 바뀐다. sequence layer 가 attention 이면 KV cache ($O(L)$) 가 그대로 남는다. 하지만 fixed-state 메모리 모듈을 쓰면 장기 세션 상태가 **weights(low-rank expert 들)** 로 대체된다 — per-user weight-delta 서빙 문제. - **sleep = 스케줄된 훈련 job**. 한 번의 sleep 은 (teacher corpus 생성 + student roll-out + LTI 완성 = 전부 generation-heavy 부하) + (reward model 호출 + 값싼 Levenshtein 채점) + (dream 마다 backward pass 1 회) + (dream 별

독립 LoRA SFT = embarrassingly parallel) + (ReST^{EM} 반복) 을 요구한다. - **sleep 1 회마다 새 모델 버전이 태어난다.** checkpoint 관리 · cache 무효화 · regression test · rollback 이 릴리스 시점의 일이 아니라 **상시운영**의 일이 된다.

비유 서버 fleet 을 데이터센터로 보면 — 낮 (wake) 에는 요청을 처리하며 로그를 쌓고, 밤 (sleep) 에는 트래픽을 끊고 **compaction · garbage collection** 을 돌린다. 단지 그 대상이 로그나 cache 가 아니라 **모델의 파라미터**일 뿐이다. “weights 의 background compaction.”

주의 (비용의 정직한 계산). step 당 비용은 Sleep 이 일반 SFT 의 약 4 배다. 대신 같은 목표 성능에 도달하는 wall-clock 으로 재면 SFT 가 3.6~4.8 배 더 걸린다. 즉 이 방식이 사는 것은 “싼 step” 이 아니라 **step · sample 효율**이다. 대가로 RL · reward model · MoE 성장부기 · 버전 관리라는 복잡도를 선불한다. 그리고 논문에는 sleep 한 사이클의 wall-clock · 에너지 분해도, 수백 사이클의 장기 배포 시뮬레이션도 없다 — 경제성은 아직 **가설**이다.

9. 성능은 실제로 나오나 (간단히)

핵심 숫자만 몇 개. (여러 결과가 표 없이 그림으로만 제시된다는 점은 감안.)

task	비교	Sleep
수학추론 AIME-24 (Qwen3-8B)	GRPO 76.4	79.2
새 언어 순차학습	ICL 은 사전학습 수준으로 붕괴	이득 유지, 단일 언어성능 거의 회복
SQuAD 지식주입 (n=200)	SEAL 43.2	46.2 (Dreaming 빼면 36.2 로 급락)
few-shot ARC	SEAL 72.5%	80%

두 가지 관찰. (1) SQuAD 에서 **Dreaming 을 빼면 성능 대부분이 사라진다** (48.9 → 35.7) — 이 task 의 이득은 대부분 dreaming 에서 온다. (2) ablation 을 자세히 보면 semantic reward 를 빼는 게 AIME-25 에서는 오히려 약간 낮다 (69.2 vs 69.0) — 외부 고정 reward model 이 늘 이롭지는 않다는 신호.

한계 (스케일의 정직한 계산). 실험의 대부분은 기성 Llama/Qwen checkpoint (1B~8B) 위에 sleep 기계를 **graft** (부착) 한 것이다. wake 와 sleep 을 **처음부터 함께 훈련**한 실증은 없다. 또 sleep 시점은 학습되지 않고 chunk 경계에 못박혀 있어, 불규칙한 실전트래픽에서의 거동은 미지수다.

10. 계보 속의 위치

이 계열을 한 줄로 요약하면: **state**는 모든 시간 규모에서 **weights**다. 토큰 단위 fast memory(Titans·Atlas), chunk 주기의 CMS level(Nested Learning), 그리고 이제 sleep 주기의 grown expert(Sleep) 까지 — 갱신 속도의 스펙트럼이 계속 넓어져 왔다. Sleep은 그 스펙트럼에 마지막으로 **가장 느린 축, 시간(잠)**을 더했다.

그리고 이 계열이 여는 질문도 바뀌었다. Titans가 “test time에 외우는 법을 배우자”로 열었던 질문은, 여섯 번째 논문에 이르러 “**모델은 언제 깨어 있고 언제 자야 하는가**”가 되었다. 그 답이 논문 한 편이 아니라 **serving 시스템의 설계** 문서처럼 생겼다는 것 — 그것이 이 계열이 시스템 엔지니어에게 남긴 초대장이다.

요약 - 문제: 계속 배우면 고정 capacity 탓에 옛것을 덮어쓴다 (catastrophic forgetting). 계속 배우는 모델에선 train/test 구분도 무의미하다. - **해법:** 뇌처럼 wake(배우기)/sleep(정리)을 교대한다. - **sleep의 두 일:** (1) **Consolidation** — 덮어쓰기 직전에 새 low-rank expert 슬롯을 열어 지식을 위로 옮기고 (upward distillation + imitation), 빠른 메모리를 비운다. (2) **Dreaming** — 스스로 연습문제를 만들어 쓸모 있는 것만 골라 다시 학습한다. - **시스템 관점:** sleep은 서빙에 붙는 스케줄된 **background 훈련 job** = “weights의 background compaction”. inference가 forward-only가 아니게 되고 (RMW 트래픽), sleep마다 새 모델 버전이 태어나며, 세션 상태가 per-user weight-delta로 바뀔 수 있다.

다음 권 예고 (G09 · 시스템 모델링 관점): 이 계열 전체를 inference 시스템의 언어로 다시 읽어, state 크기 · 토큰당 연산 · 메모리계층 · 서빙 경제학을 하나의 cost model로 묶는다.